

Function Approximation: From Linear Models to Neural Networks

Scientific Machine Learning

Alena Kopaničáková

Toulouse INP / ENSEEIHT

Spring 2026

Outline

- 1 Introduction to SciML
- 2 Supervised Learning Framework
- 3 Linear Models
- 4 Polynomial Basis Models

Course Organization

9 Lectures (CTD)

Theory + worked examples in class

Each lecture builds on the previous:
approximation $\rightarrow$ PINNs $\rightarrow$ operators $\rightarrow$ hybrid

6 Lab Sessions (TP)

Hands-on implementation in Python / PyTorch

- Code models from scratch
- Run experiments, interpret results
- Reproduce key theoretical findings

Final Exam

Balanced between:

Theory - derivations, error analysis, proofs, conceptual questions

Practice - design models, interpret results, choose architectures, pseudocode, discuss practical difficulties

What is Scientific Machine Learning?

SciML sits at the intersection of machine learning, numerical analysis, applied mathematics and optimization. Its goal: learn accurate, fast computational models of physical systems from data and physical structure.

Classical Simulation

- Physics-based, first principles
- Mesh / discretization required
- Rigorous error bounds, mature theory
- Expensive: minutes to hours per run

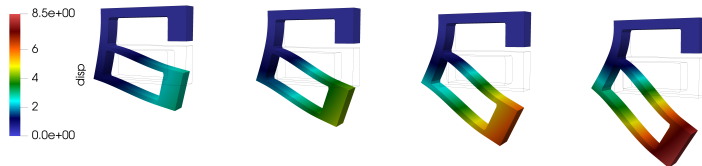
Pure Machine Learning

- Fully data-driven; no physics
- Fast inference after training
- No built-in conservation or symmetry
- Needs large labeled datasets

SciML: Combining Both Worlds

Encode physical structure *inside* ML models - faster than solvers at query time, more data-efficient and reliable than black-box ML

Parametric partial differential equations (PDEs)



Let $\Omega \subset \mathbb{R}^d$, $\mathbf{A} \subset \mathbb{R}^P$, and $\mathcal{V} = \mathcal{V}(\Omega)$ be a Hilbert space. The parametric problem is given in abstract strong form as:

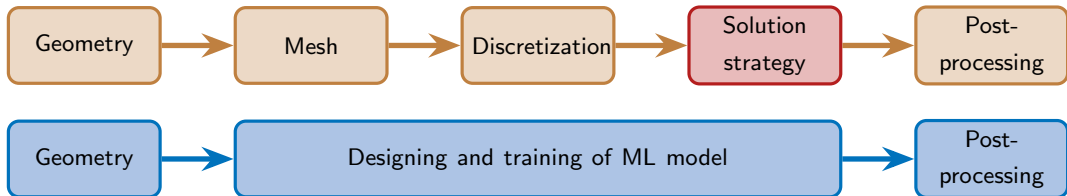
For a given $\alpha \in \mathbf{A}$, find the solution $u(\alpha) \in \mathcal{V}$ such that

$$\mathcal{P}(\alpha)u(\alpha) = f(\alpha), \quad \text{in } \mathcal{V}',$$

where $\mathcal{P}(\alpha) : \mathcal{V} \rightarrow \mathcal{V}'$ is a differential operator and $f(\alpha)$ is a linear continuous form

- Applications areas: uncertainty quantification, design optimization, or inverse problems, ...
- Problem can be parametrized, for example, wrt. material parameters, source term, or boundary conditions, ...

Solution of a PDE



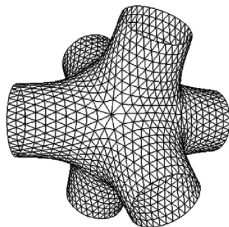
“Classical” numerical methods

- **High-fidelity** solutions using advanced methods from numerics, scientific computing, optimization, ...
- Capture physics on multiple scales
- Capture well the details and **high-frequency** features
- Encounter challenges in data integration

Scientific machine-learning (SciML)

- Exceptionally cheap **low-fidelity surrogates** using novel machine-learning (ML)
- Discover systems dynamics
- Capture well global trends and **low-frequency** features (spectral bias)
- Seamless integration of data

“Classical” numerical methods - high-fidelity solution

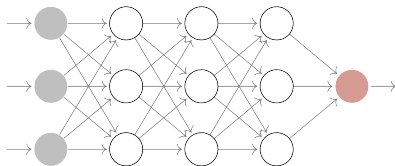


High-fidelity discretizations

- Finite elements/Finite differences/...
 - Requires meshing
- High accuracy, but also high computational cost, especially as the problem size grows
- Resolves well high-frequency features

Solution strategies

- Discretization gives rise to the linear/non-linear system of equations
- Various type of linear (from direct to iterative) and nonlinear (1st, 2nd order) solution strategies can be employed
- The most efficient and robust solvers for large-scale problems typically utilize divide and conquer approaches, e.g., multilevel, domain-decomposition, field-split, ...



Key idea

Use ML to enhance the numerical simulations, e.g.,

- Process and analyze large volume of data, which can be then used for example as BC/IC
- Mesh design and adaptivity
- **Approximate solution of PDEs using DNNs**
- Initialization of the high-fidelity numerical solvers
- Replacing specific parts of the high-fidelity simulation codes
- Pre/post-processing, visualization and results interpretation
- ...

SciML: Approximate solution of PDEs

Physics-informed learning (PINNs)

- Focus on a **single instance PDE**
- Integration of the data and physics
- "Mesh-free" (complex geo, high-dim. problems)
- Discovery of physics and model parameters
- Initial related work^[1,2,3]

Operator learning

- Model function-to-function relations
- Focus on a **family of parametric PDEs**
- Generalization across parameter spaces
- Initial related work^[4,5,6]

[1] Dissanayake, Phan-Thien. "Neural-network-based approximations for solving partial differential equations." 1994.

[2] Lagaris, Isaac E., Aristidis Likas, and Dimitrios I. Fotiadis. "Artificial neural networks for solving ordinary and partial differential equations." 1998.

[3] Raissi et al. "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations." 2017.

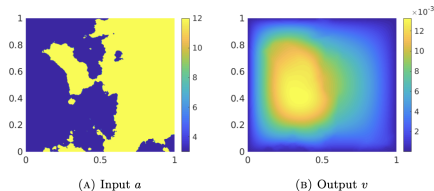
[4] Chen, Chen. "Universal approximation to nonlinear operators by neural networks with arbitrary activation functions and its application to dynamical systems." 1995.

[5] Lu et al. "Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators." 2019.

[6] Li, Zongyi, et al. "Fourier neural operator for parametric partial differential equations." 2020.

Example 1: Darcy PDEs

$$\begin{aligned} -\nabla(a\nabla u) &= f, & u &\in \Omega \\ u &= 0, & u &\in \partial\Omega. \end{aligned}$$



Quantities of interest are:

- u is temperature or pressure
- a is conductance or permeability
- f is source

Examples of many-query problem

Classical solvers are accurate but slow. One high-fidelity run: minutes to hours.

Settings that demand many runs

- **Design optimization:** Evaluate drag/lift for $\sim 10^4$ geometries
- **Uncertainty quantification:** Propagate a distribution over uncertain inputs
- **Real-time control:** Need $u(\mu)$ at actuator rate, not solver rate
- **Inverse problems:** Forward map queried $\sim 10^6 \times$ inside MCMC

The Bottleneck

Many-query setting: 10^3 - 10^6 solves required. At hours per run - **infeasible**

ML Solution: Amortize the Cost

Train surrogate $\mathcal{G}_\theta \approx \mathcal{G}^\dagger$ *once* (offline). Query in milliseconds (online). Cost shared over all future queries

Three SciML Paradigms

Physics-Informed Learning (PINNs)

Embed the PDE residual as a soft penalty in the training loss $\implies$ **no mesh, data optional**

Neural Operators (NO)

Learn $\mathcal{G}_\theta \approx \mathcal{G}^\dagger$ from input-output pairs (a_n, u_n) produced by a classical solver $\implies$ **no PDE required at inference**

Hybrid Methods

Neural network components embedded inside a classical numerical solver $\implies$ Retains stability guarantees; network corrects solver errors

This Course at a Glance

Module 1: Function Approximation

Hypothesis class, bias-variance trade-off, linear/Fourier/NN models, UAT, SGD

Module 3: Operator Learning

DeepONet, Fourier Neural Operator, operator approximation theory

Module 2: Physics-Informed NNs

PINN loss, collocation, convergence, failure modes, extensions, inverse problems, equation discovery

Module 4: Hybrid Methods

ML-accelerated solvers, learned preconditioners, neural closures

The Supervised Learning Problem

Setting: Dataset of N labeled examples

$$\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N, \quad \mathbf{x}_i \in \mathbb{R}^d, \quad y_i \in \mathbb{R},$$

generated by an unknown target $f^* : \mathbb{R}^d \rightarrow \mathbb{R}$ with noise:

$$y_i = f^*(\mathbf{x}_i) + \varepsilon_i, \quad \varepsilon_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, \sigma^2)$$

Goal: Construct $\hat{f} : \mathbb{R}^d \rightarrow \mathbb{R}$ that *generalizes* to unseen inputs

Context: Scientific Machine Learning

- Surrogate models for parametric PDEs
- Data-driven closure terms
- Operator learning architectures (FNO, DeepONet)

Hypothesis Class

The central design choice in supervised learning:

Definition: Hypothesis Class

A parameterized family of candidate functions:

$$\hat{\mathcal{F}} = \{ f_{\boldsymbol{\theta}} : \mathbb{R}^d \rightarrow \mathbb{R} \mid \boldsymbol{\theta} \in \Theta \subseteq \mathbb{R}^p \}$$

Small $\hat{\mathcal{F}}$ (few parameters p)

- Easy to optimize
 - Generalizes from limited data
 - May not approximate f^* well
- ⇒ **High bias / underfitting**

Large $\hat{\mathcal{F}}$ (many parameters p)

- Approximates f^* closely
 - Sensitive to training data
 - Harder to optimize
- ⇒ **High variance / overfitting**

Empirical and Population Risk

We measure quality with the **squared error loss**

Population Risk (True Objective)

$$\mathcal{R}(\hat{f}) = \mathbb{E}_{\mathbf{x}} \left[\left| \hat{f}(\mathbf{x}) - f^*(\mathbf{x}) \right|^2 \right]$$

Cannot be computed directly - requires the distribution of inputs

Empirical Risk (Training Loss)

$$\hat{\mathcal{R}}(\hat{f}) = \frac{1}{N} \sum_{i=1}^N \left| \hat{f}(\mathbf{x}_i) - y_i \right|^2$$

Feasible proxy - what we actually minimize

Key challenge: the *generalization gap* $\mathcal{R}(\hat{f}) - \hat{\mathcal{R}}(\hat{f})$

A model can achieve $\hat{\mathcal{R}} = 0$ while $\mathcal{R}$ remains large: **overfitting**

Bias-Variance-Noise Decomposition

Fix any point $\mathbf{x}$, the expected squared prediction error decomposes:

Proposition (BVN Decomposition)

$$\mathbb{E}_{\mathcal{D}} \left[\left| \hat{f}(\mathbf{x}) - y_{\text{new}} \right|^2 \right] = \underbrace{\left(\mathbb{E}_{\mathcal{D}}[\hat{f}(\mathbf{x})] - f^*(\mathbf{x}) \right)^2}_{\text{Bias}^2} + \underbrace{\text{Var}_{\mathcal{D}}[\hat{f}(\mathbf{x})]}_{\text{Variance}} + \underbrace{\sigma^2}_{\text{Noise}}$$

Bias²

- systematic error
- zero only if $f^* \in \hat{\mathcal{F}}$
- decreases* as $\hat{\mathcal{F}}$ grows

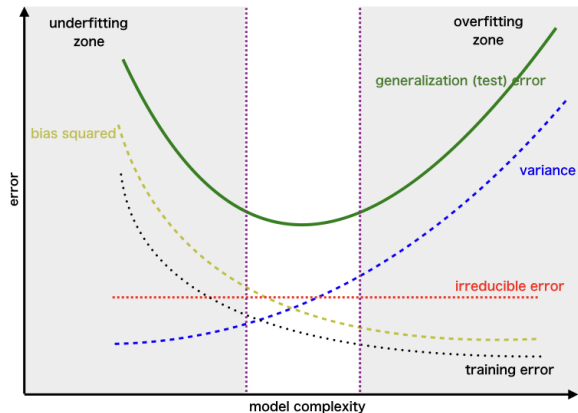
Variance

- sensitivity to training data
- increases* as $\hat{\mathcal{F}}$ grows (typically)

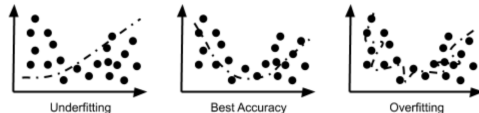
Noise σ^2

- irreducible
- no estimator can beat this floor

The Bias-Variance Trade-off Illustrated



- **Underfitting:** model too simple, high bias, misses structure
- **Best fit:** optimal complexity, best generalization
- **Overfitting:** model too complex, high variance, fits noise



Three-Term Error Decomposition

A complementary, *algorithmic* decomposition of the generalization error:

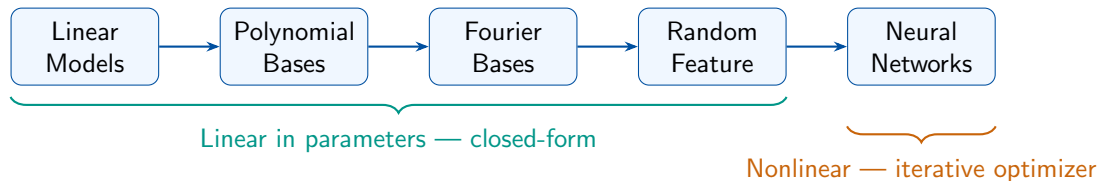
Theorem

$$\mathcal{R}(\hat{f}) \leq \underbrace{\mathcal{R}(f_G^*)}_{\text{approximation error}} + \underbrace{2 \sup_{\hat{g} \in \hat{\mathcal{F}}} |\mathcal{R}(\hat{g}) - \hat{\mathcal{R}}(\hat{g})|}_{\text{generalization gap}} + \underbrace{\hat{\mathcal{R}}(\hat{f}) - \hat{\mathcal{R}}(f_T^*)}_{\text{optimization error}}$$

Term	Meaning	Cause	Remedy
Approximation	Can $\hat{\mathcal{F}}$ represent f^* ?	$\hat{\mathcal{F}}$ too small	Enlarge model
Generalization	Training $\approx$ Testing?	N too small	More data; regularize
Optimization	Minimized $\hat{\mathcal{R}}$ well?	Poor optimizer	More epochs; better schedule

A Progressive Hierarchy of Models

We progressively enlarge $\hat{\mathcal{F}}$ from linear functions to neural networks:



For each model family we derive:

- 1 The model and its training procedure
- 2 The three error terms: approximation, generalization gap, optimization

Linear Models: Model and Normal Equations

Hypothesis class: $\hat{\mathcal{F}} = \{\mathbf{x} \mapsto \boldsymbol{\theta}^\top \mathbf{x} \mid \boldsymbol{\theta} \in \mathbb{R}^d\}$, $p = d$ parameters.

Empirical risk: With design matrix $\mathbf{X} \in \mathbb{R}^{N \times d}$, target $\mathbf{y} \in \mathbb{R}^N$:

$$\hat{\mathcal{R}}(\boldsymbol{\theta}) = \frac{1}{N} \|\mathbf{X}\boldsymbol{\theta} - \mathbf{y}\|_2^2 \quad (\text{Convex quadratic})$$

Normal Equations

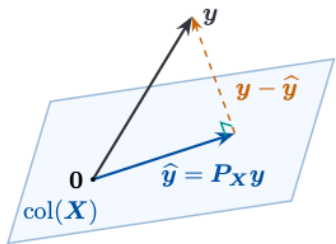
$$\mathbf{X}^\top \mathbf{X} \boldsymbol{\theta}^* = \mathbf{X}^\top \mathbf{y} \quad \Rightarrow \quad \boldsymbol{\theta}^* = (\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y},$$

when $\mathbf{X}$ has full column rank ($N \geq d$)

Linear Models: Geometric Interpretation

Orthogonal Projection

Predictions $\hat{\mathbf{y}} = \mathbf{P}_X \mathbf{y} = \mathbf{X}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \mathbf{y}$ are the **orthogonal projection** of $\mathbf{y}$ onto $\text{col}(\mathbf{X})$. Residual $\mathbf{y} - \hat{\mathbf{y}}$ is orthogonal to every column of $\mathbf{X}$.



- $\hat{\mathbf{y}}$ is the **closest point** in $\text{col}(\mathbf{X})$ to $\mathbf{y}$
- Residual $\mathbf{y} - \hat{\mathbf{y}} \perp \text{col}(\mathbf{X})$
- Least-squares finds the best approximation of $\mathbf{y}$ in $\text{col}(\mathbf{X})$

Underdetermined case ($N < d$)

$\mathbf{X}^\top \mathbf{X}$ singular: use the **Moore-Penrose pseudoinverse** $\theta^+ = \mathbf{X}^+ \mathbf{y}$ (minimum-norm solution).

Linear Models: Ridge Regularization

Limitations of plain linear models:

- High bias: cannot capture any nonlinear structure
- Irrelevant features: All d input dimensions contribute with fixed linear weights
- Underdetermined ($N < d$): infinitely many solutions
- Ill-conditioned: small eigenvalues of $\mathbf{X}^\top \mathbf{X}$ amplify noise variance

Ridge Regression (Tikhonov Regularization)

$$\theta_\lambda^* = (\mathbf{X}^\top \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^\top \mathbf{y}, \quad \lambda > 0$$

Linear Models: Error Analysis Summary

Error term	Value	Comment
Approximation	> 0 if $f^* \notin \hat{\mathcal{F}}$	Fixed by model family; <i>does not</i> improve with more data
Generalization gap	$\mathcal{O}(d/N)$	Shrinks with data; controlled by Ridge ($d \rightarrow d_{\text{eff}}$)
Optimization	$= 0$	Exact closed-form via normal equations

Key Insight

Linear models achieve the **best possible optimization error** (zero) but pay a **high approximation cost** for any nonlinear f^* . As we enlarge $\hat{\mathcal{F}}$: approximation error $\downarrow$ but optimization error $\uparrow$

Polynomial Basis Models: Idea

Key idea: apply a nonlinear *feature map* $\phi : \mathbb{R}^d \rightarrow \mathbb{R}^M$, then fit linearly.

For $d = 1$ and degree m : $\phi_k(x) = x^k$, $k = 0, \dots, m$:

$$\hat{f}(x) = \boldsymbol{\theta}^\top \boldsymbol{\phi}(x) = \sum_{k=0}^m \theta_k x^k, \quad p = m + 1.$$

Design matrix (Vandermonde):

$$\boldsymbol{\Phi} = \begin{pmatrix} 1 & x_1 & \cdots & x_1^m \\ \vdots & & & \vdots \\ 1 & x_N & \cdots & x_N^m \end{pmatrix} \in \mathbb{R}^{N \times M}$$

Linear in Parameters

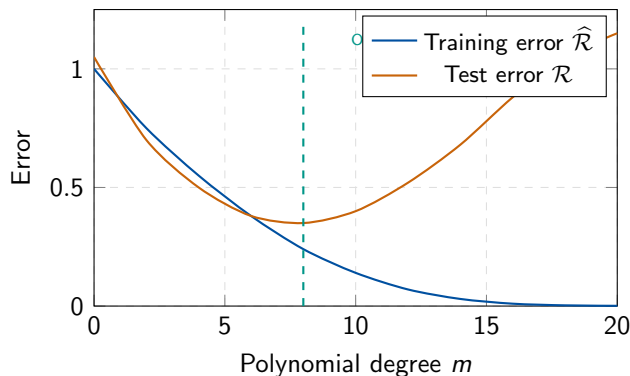
Despite x^k nonlinearity, model is **linear in $\boldsymbol{\theta}$** .

$\Rightarrow$ Same normal equations:

$$\boldsymbol{\Phi}^\top \boldsymbol{\Phi} \boldsymbol{\theta}^* = \boldsymbol{\Phi}^\top \mathbf{y}$$

Closed-form solution available.

Polynomial Models: Bias-Variance Trade-off



As degree m grows:

- **Bias** $\downarrow$: with $N = m + 1$, exact interpolation possible
- **Variance** $\uparrow$: Runge's phenomenon - wild oscillations
- **Conditioning** $\downarrow$: $\kappa(\Phi^\top \Phi)$ grows rapidly

Common Pitfall

Minimizing training error always favors larger m since $\hat{\mathcal{R}} \rightarrow 0$ as $m \rightarrow N$.

Use cross-validation!

Polynomial Models: Error Analysis

Three Error Terms (Degree- m Polynomial, $f^* \in C^s$)

- **Approximation:** $\mathcal{O}(m^{-2s})$ (Weierstrass: can drive to zero by increasing m)
- **Generalization gap:** $\mathcal{O}((m+1)/N)$, (increases linearly with m)
- **Optimization:** $= 0$ (exact via normal equations)

Optimal degree (balance approximation vs. generalization):

$$m^* \asymp N^{\frac{1}{2s+1}}, \quad \mathcal{R}(\hat{f}_{m^*}) = \mathcal{O}\left(N^{-\frac{2s}{2s+1}}\right).$$

Smoother f^* (larger s) $\Rightarrow$ faster convergence.

Curse of Dimensionality

For $f^* \in C^s([0, 1]^d)$ with $d > 1$: $\mathcal{R}(\hat{f}) = \mathcal{O}\left(N^{-\frac{2s}{2s+d}}\right)$.

For $s = 2$, $d = 10$: exponent ≈ 0.29 vs. 0.80 in 1D.